Project

What's expected

- All projects should be developed locally but should be deployed on the clusters[1] you used during the semester (ideally using k8s).
- Clusters will be available with reservations (using the provided website) during reading period.
- You can use the technologies we discussed during the semester (Postgres, MongoDB, K8s, Minio S3) or others may be used after discussing with your instructor.
- Each group member should have equal contributions to the project.
- All orchestration should be implemented using Apache Airflow running on the clusters.
- Each source file (DAGs and any additional files, like yaml files) must include author information.
  - We will use that to conduct the defense.
- Follow the best practices discussed during class for your project.

Submission

- A document with the requirement specification for your pipeline. I.e., what downstream needs is the pipeline supporting.
- All code and configuration files, including setup instructions, must be submitted to the group repository (ADTG).
- A document describing the pipeline design, DAGs, trigger properties, etc., as well as the justification for the technologies used (advantages to others visited during the course).
- Your project should run on the cluster[2].
- Project should **not** include the downstream processing, only the pipeline that would support those (ideally each team member will contribute to one of them).

Defense

- You should get your project ready for the defense the day before.
- During the defense, we will ask questions to the team members individually, focusing to what they implemented.
- Each student will receive the points for the group upon a successful project defense.
  - In case the defense is not successful (e.g., not be able to articulate the answers/explanations), the student may not receive full marks, with the possibility of receiving no marks at all.

#5 — Real-time trade analysis

The objective of this project is to design and implement a data pipeline that integrates two market data sources from Finnhub: a **real-time trade stream** delivered through the WebSocket API and a **company news feed** delivered through the company news endpoint. The trade stream provides live trade updates for subscribed symbols, while the news endpoint provides recent company-related articles that can be retrieved for a specified ticker and date range.

Your team will build an end-to-end pipeline that receives trade events continuously, ingests company news periodically, and produces curated datasets for downstream analysis. The key requirement in this project is that the downstream application must use **both** sources together. The purpose is not only to collect market activity and news independently, but to support analyses that relate price or trading behavior to the timing and content of company-specific news.

Because the sources operate at different speeds and granularities, the pipeline should be designed to handle continuous event ingestion for trades and scheduled retrieval for news. You should also anticipate ambiguity in the data, including duplicated news articles, delayed publication, out-of-order trade messages, missing fields, and difficulty defining exactly which trades should be considered “related” to a given news item. The design should allow additional symbols, broader watchlists, or new streaming event types without major redesign, the data model should support adding sentiment scores or article classification in later stages

Examples of Downstream Data Requirements

The curated data should support downstream processes that explicitly combine streaming trades with company news:

1. **News-to-Market Reaction Analysis:** support analysis of trading behavior before and after a news event, enable comparison of price movement and volume around publication time
2. **Event-Centered Windows:** provide a mechanism to group trades into time windows surrounding each news article, such as 15 minutes before publication and 60 minutes after publication, the exact window definition may be treated as an assumption
3. **Abnormal Activity Detection:** enable downstream logic to identify unusual price changes or trading volume associated with recent news, support simple baseline comparisons against prior intra-day activity
4. **Unified Company Event View:** provide a company-level view that combines trade summaries and recent news events, allow analysts to query all relevant market and news activity for a ticker over a selected period
5. **Screening and Monitoring:** support dashboards or alerts for symbols with both high news activity and strong market reaction, allow flexible thresholds for what constitutes "high" activity
6. **Historical Replay and Backtesting:** preserve enough raw and curated data to reconstruct how the market reacted to past news events, support exploratory work on simple event-driven trading or monitoring strategies

- API sources:
  - trades
  - news

API links:

1. https://finnhub.io/docs/api/websocket-trades
2. https://finnhub.io/docs/api/company-news